PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Условный оператор if / elif / else

Содержание

- Зачем нужны условия
- Синтаксис if / elif / else
- Составные условия
- Операторы сравнения
- Важные правила

Зачем нужны условия

Привет! На этом занятии ты научишься объяснять компьютеру, как принимать решения. Представь: ты разрабатываешь систему, которая определяет, кому положена скидка, кого пустить на мероприятие или к какой категории относится товар. Человек читает условия и понимает их смысл. Программа — нет. Она проверяет значения переменных и выбирает ветку кода.

Именно это умеет условный оператор `if`. Из нескольких таких условий можно составить простой **классификатор по правилам**: программа относит объект к категории по заданным признакам. Это ещё не машинное обучение — правила здесь пишет человек. Но так легче понять, что делает модель ИИ: она тоже относит объекты к классам, только закономерности находит сама по данным.

После этого занятия ты сможешь:

- писать цепочки `if` / `elif` / `else`;
- составлять условия с `and`, `or`, `not`;
- создавать программу, которая определяет категорию объекта по его признакам.

Синтаксис if / elif / else

Условный оператор выбирает, какой блок кода выполнить, в зависимости от значения переменной.
Пример программы:

```
temperature = float(input('Введите температуру: '))


if temperature > 37.5:
    print('Повышенная температура')
elif temperature < 36.0:
    print('Пониженная температура')
else:
    print('Норма')
```

> **Внимание**
>
> Двоеточие в конце строки с условием и отступ в 4 пробела — обязательны.

Обратите внимание на граничные значения. Условия записаны строго: `> 37.5` и `< 36.0`. Поэтому при вводе ровно 37.5 или ровно 36.0 ни одно из них не выполнится, и программа выведет «Норма». Если граничное значение должно относиться к другой категории, используйте `>=` или `<=`.
Ввод и тип данных. Функция `input()` всегда возвращает строку. Чтобы сравнивать введённое значение с числом, его преобразуют: `int()` — в целое число, `float()` — в дробное. Если пользователь введёт не число (например, «тридцать»), `float()` завершится ошибкой `ValueError` и программа остановится. Поэтому вводите данные в том формате, который указан в задании.
Веток `elif` может быть несколько. Python проверяет их сверху вниз и выполняет **только первую** ветку с истинным условием — остальные уже не проверяются:

```
score = 75

if score >= 90:
    print('Отлично')
elif score >= 70:
    print('Хорошо')
elif score >= 50:
    print('Удовлетворительно')
else:
    print('Неудовлетворительно')
# Выведет: Хорошо
```

При `score = 75` условие `score >= 50` тоже истинно, но эта ветка не выполнится: Python уже выбрал ветку «Хорошо».

Составные условия

Любое условие в Python даёт одно из двух **логических значений**: `True` (истина) или `False` (ложь). Например, `5 > 3` — это `True`, а `2 == 7` — `False`. Логические значения можно хранить и в переменных: `has_ticket = True`, `is_blocked = False`. Такой тип данных называется `bool`.
Иногда нужно проверить сразу несколько условий. Для этого используются логические операторы:

Оператор	Смысл	Пример
`and`	Оба условия истинны	`age >= 18 and has_ticket`
`or`	Хотя бы одно истинно	`score < 0 or score > 100`
`not`	Инверсия (отрицание)	`not is_blocked`

Переменная `has_ticket` уже хранит `True` или `False`, поэтому сравнивать её с `True` не нужно: записи `has_ticket` и `has_ticket == True` дают одинаковый результат, но первая короче и понятнее.
Порядок выполнения. Если в одном условии несколько логических операторов, Python сначала выполняет `not`, затем `and`, затем `or`. Чтобы условие читалось однозначно, расставляйте скобки:

```
age = 15
has_parent = False
is_vip = True

# and выполняется раньше or:
print(is_vip or age >= 18 and has_parent)    # True  — то же, что is_vip or (age >= 18 and has_par
print((is_vip or age >= 18) and has_parent)  # False — скобки изменили порядок
```

Операторы сравнения

Оператор	Значение	Оператор	Значение
`==`	равно	`!=`	не равно
`>`	больше	`<`	меньше
`>=`	больше или равно	`<=`	меньше или равно

> **Внимание**
>
> `=` — это присваивание (записать значение в переменную), а `==` — сравнение (проверить, равны ли значения). Запись `if x = 5:` — синтаксическая ошибка, правильно `if x == 5:`.

Важные правила

- После `if` / `elif` / `else` всегда стоит двоеточие.
- Тело блока сдвигается на 4 пробела вправо (отступ).
- Python проверяет ветки сверху вниз и останавливается на первой истинной.
- Порядок `elif` важен: ставьте более специфичные условия раньше.
- `else` — «запасная» ветка, она выполняется, если ничего не сработало.

Главное правило: Python проверяет ветки сверху вниз и останавливается на первой истинной. Поэтому порядок условий имеет значение!

Материалы к занятию

- Презентация к занятию 1.6 «Условия в Python: принятие решений»